
OpenVela 工程实践 • 上机实验

小智 AI 语音助手

手册章节 综合实验

DshanPI openvela Devkit • T113S3

姓 名张恒基

学 号2024210926

班 级2024211301

指导教师修佳鹏

提交日期2026 年 06 月 19 日

参考：《openvela 快速入门与工程实践（基于 T113S3）》

摘要

本报告记录小智 AI 语音助手在 DshanPI openvela 开发板上的完整联调过程。基础任务完成多进程协同框架接入、Wi-Fi 联网、设备激活与语音问答；进阶任务在 STT 识别文本上增加本地关键词解析，实现「开灯 / 关灯」语音控制 LED1。报告结合操作步骤、原理分析与实验截图，说明从配置切换到端到端语音链路的全流程。

关键词：小智；WebSocket；Opus；ASR；LLM；TTS；语音控制；LED

目录

一、实验目的	1
二、实验环境	1
三、实验原理	3
3.1 设备端多进程协同	3
3.2 云端处理链路	3
3.3 进阶 LED 控制的两种路径	4
3.4 WebSocket 消息类型（板端视角）	4
3.5 xiaozhi.sh 启动链	5
3.6 leds.c 与 GPIO 电平约定	5
3.7 编译打包与 usrddata	5
四、实验内容与操作步骤	5
4.1 （一）基础程序检查与配置确认	5
4.2 （二）小智语音助手基础流程实现	7
4.3 （三）进阶功能：语音控制 LED	8
4.4 （四）编译、打包和结果确认	9
4.5 （六）相关实验截图与实机联调	10
4.6 联调分段检查清单	16
五、端到端流程图	17
六、关键源码、烧录流程与排错记录	19
6.1 xiaozhi.sh 自启动脚本（板端 /data/xiaozhi.sh）	19
6.2 STT 分支挂接与语音控灯函数	19
6.3 LED 底层 leds.c	20
6.4 defconfig 与 rcS 切换命令	20
6.5 编译、打包、烧录	21
6.6 问题现象、原因与修改	21
七、实验结果汇总	22
八、实验总结	23

一、实验目的

本次综合实验围绕 openvela 平台上的小智 AI 语音助手展开，目标是让开发板能够通过 Wi-Fi 连接小智服务器，完成语音上传、服务器识别和语音回复播放。通过本次实验，我主要熟悉了语音助手在开发板端的程序结构、网络连接、音频采集播放、WebSocket 通信和界面显示流程。

在进阶部分，我在原有语音助手程序基础上增加了语音控制 LED 的功能。用户在正常对话中说出「开灯」或「关灯」时，程序能够根据服务器返回的语音识别文本直接控制开发板 LED1 亮灭，从而把语音交互和 GPIO 外设控制结合起来。

具体而言，实验希望达成以下目标：

1. 理解设备端多程序协同架构：弄清 wifi_manager、arecord、aplay、control_center、lvgl_demo 各自职责，以及 /data/xiaozhi.sh 启动脚本如何把各模块串联起来；
2. 完成工程配置切换与镜像集成：将课程资料包中的综合实验配置（defconfig、rcS、UDISK 资源）正确合入当前 SDK，使实验三应用真正编入系统镜像，而非停留在实验二的 MyWhackMole 配置；
3. 实现与小智服务器的联网配对：开发板经 Wi-Fi 接入互联网，在 xiaozhi.me 完成设备激活，建立 WebSocket 长连接；
4. 验证语音问答端到端流程：对着板载麦克风说话，观察 Opus 音频上行、服务器 ASR/LLM/TTS 处理、下行语音播放与界面状态刷新；
5. 进阶：语音控制 LED：在服务器返回的 STT 文本中本地匹配「开灯」「关灯」等关键词，直接驱动 LED1 亮灭，将语音语义与 GPIO 外设控制衔接起来。

与初探实验（任务/线程）、基础实验（LVGL 图形游戏）相比，综合实验更强调系统级集成：问题往往不出在某一行业务代码，而出在 defconfig 是否启用、rcS 是否自启动、资源是否打进 usrdata、多个后台任务是否同时存活。

二、实验环境

项目	配置
实验平台	DshanPI openvela Devkit (T113S3)
主机系统	Windows 11
虚拟机环境	VMware + Ubuntu 22.04
源码目录	~/vela-opensource/
开发工具	VS Code 远程 SSH 连接 Ubuntu

项目	配置
编译目录	~/vela-opensource/vendor/allwinnertech/lichee/
编译命令	source vela_env.sh → source envsetup.sh → lunch_nuttx → m → pack
烧录工具	PhoenixSuit; 镜 像 rtos_nuttx_r528s3-velaevb1_uart0_256Mnand.img
串口工具	MobaXterm, Serial, 1500000 baud, Flow Control: none
网络	开发板 Wi-Fi STA 连接手机热点或实验室 AP
云端服务	小智服务器 (xiaozhi.me 激活与管理)
主要程序	control_center、wifi_manager、arecord、aplay、lvgl_demo

两根 USB 线分别承担烧录调试与串口日志：串口是联调时观察 Opus 帧计数、WebSocket 连接状态、STT 文本与 LED 控制日志的主要窗口。开发板通过两根 USB 线连接 Windows 主机：一根用于 PhoenixSuit 烧录，另一根枚举为串口（本机常见 COM5，CH343），MobaXterm 以 1500000 波特率连接 NSH。源码放在 Ubuntu 虚拟机 ~/vela-opensource/，VS Code Remote-SSH 直接编辑；编译入口固定为 vendor/allwinnertech/lichee/，切勿在未 source envsetup.sh 时执行 m。

主要路径速查：

用途	路径
control_center	apps/examples/control_center/control_center.c
LED 驱动封装	apps/examples/control_center/leds.c
Wi-Fi 管理	apps/examples/wifi_manager/
界面程序	apps/examples/lvgl_demo/
开机脚本	board/common/data/UDISK/xiaozhi.sh → 板端 /data/xiaozhi.sh
板级 defconfig	vendor/allwinnertech/boards/r528/r528s3-velaevb1/configs/nsh/defconfig
rcS 自启动	板级 init.d/rcS 末尾 sh /data/xiaozhi.sh &
资料包参照	综合实验 9-5_系统集成/
编译产物	out/r528s3/velaevb1_nand/ rtos_nuttx_r528s3-velaevb1_uart0_256Mnand.img

三、实验原理

3.1 设备端多进程协同

(1) 小智 AI 语音助手的设备端由多个程序协同完成。系统启动后，rcS 脚本运行 `/data/xiaozhi.sh`，依次启动 `wifi_manager`、`arecord`、`aplay`、`control_center` 和 `lvgl_demo`。这样开发板启动后可以自动进入语音助手运行环境，不需要每次手动逐个运行程序。若 rcS 中该行被注释，或 `xiaozhi.sh` 未打入 `usrdata`，烧录后只能看到 NSH 或单一界面，无法形成完整语音助手。

小智语音助手在开发板上并非单一可执行文件，而是由多个 NuttX 应用分工协作：

1. `wifi_manager`：负责读取 `/data/wifi.cfg` 或交互式配置，拉起 `wlan0`，为后续联网提供 IP；
2. `arecord`：从麦克风采集 PCM，按帧 Opus 编码，将编码后的音频块交给 `control_center` 或经 IPC 发送；
3. `aplay`：接收服务器下发的 TTS 音频流，经声卡解码播放；
4. `control_center`：核心枢纽——维护 WebSocket 连接、上传 Opus、解析 JSON 控制消息（hello、stt、tts、iot 等）、协调各模块状态；
5. `lvgl_demo`：LVGL 界面，显示激活码、连接状态、识别文本与对话动画。

系统启动后，`/etc/init.d/rcS` 末尾执行 `sh /data/xiaozhi.sh &`，脚本内按顺序拉起上述进程。若 rcS 中该行被注释，或 `xiaozhi.sh` 未被打包进 `usrdata`，则烧录后需手工逐个启动，极易遗漏某一环导致「能联网但没声音」或「有界面但不上传音频」。

3.2 云端处理链路

(2) 用户对开发板讲话后，`arecord` 从麦克风采集 PCM 音频，并经过 Opus 编码后把音频数据交给 `control_center`。`control_center` 通过 WebSocket 把音频上传到小智服务器，同时接收服务器返回的控制消息、识别文本和语音数据。

(3) 小智服务器收到音频后，会调用 ASR 模块把语音转换成文本，再把文本交给 LLM 生成回复。回复文本经过 TTS 模块合成为语音后，由服务器通过连接返回给开发板。开发板端 `aplay` 接收到音频数据后通过声卡播放出来，用户就能听到大模型的语音回答。用户对开发板讲话后，数据流大致为：

1. `arecord` 采集 PCM 并 Opus 编码；`control_center` 经 WebSocket 将二进制音频帧发送至小智服务器；服务器 ASR 模块将语音转为文本（STT）；LLM 根据用户文本与对话历史生成回复文本；TTS 将回复合成为语音（常见为 Opus 或 PCM 流）；

服务器经同一 WebSocket 回传音频与状态 JSON； aplay 播放下行音频，lvgl demo 刷新界面。

小智开源服务端 (xiao zhi-esp32-server) 支持 WebSocket、MQTT+UDP、流式 ASR、流式 TTS、LLM、Intent 与客户端 IoT 协议。本实验开发板端主要使用 WebSocket 和 Opus 音频流完成语音通信，同时利用 IoT 协议和本地 STT 文本判断完成 LED 控制。

用户对开发板讲话后的时序可概括为：麦克风采集 → Opus 编码 → WebSocket 上行 → 服务器 ASR 得文本 → LLM 生成回复 → TTS 合成 → WebSocket 下行 → aplay 播放。任一环节断开，都会表现为「能亮屏不能听」「能联网不能答」等部分功能缺失，联调时需分段定位。

3.3 进阶 LED 控制的两种路径

(4) 小智服务器项目支持 WebSocket、MQTT+UDP、流式 ASR、流式 TTS、LLM、Intent 和客户端 IoT 协议。本实验开发板端主要使用 WebSocket 和 Opus 音频流完成语音通信，同时利用 IoT 协议和本地 STT 文本判断完成 LED 控制。

原工程已支持云端 IoT 命令：服务器返回 `type: iot` 且 `commands` 中含 LED1 的 `SetStatus` 时，`control_center` 调用 `leds_ctl`。题目要求用户直接说「开灯」「关灯」即可动作，因此在 STT 分支增加本地 `strstr` 关键词匹配，在 `send_stt` 显示文本的同时触发 GPIO，无需额外唤醒词，也不阻塞 TTS 播放。

`leds.c` 中 LED1 映射 `/dev/gpio00`，写 0 点亮、写 1 熄灭（低电平有效），与基础实验 `MyWhackMole` 的 `flash_led1` 一致。

3.4 WebSocket 消息类型（板端视角）

`control_center` 与服务器之间除二进制 Opus 帧外，还有 JSON 文本消息。联调时常见 `type` 包括：

type	含义与板端处理
hello	握手与能力协商；连接建立后首包
stt	ASR 识别文本；送 lvgl demo 显示，进阶分支控 LED
tts	TTS 状态（开始/结束）；协调 aplay 与 UI 动画
iot	云端设备控制；解析 <code>commands</code> 调用 <code>leds_ctl</code> 等
llm	大模型回复文本；可选显示在界面

理解消息分流后，可快速定位「能上网、能编码、但灯不亮」是 STT 未命中关键词，还是 IoT 未下发。

3.5 xiaozhi.sh 启动链

`/data/xiaozhi.sh` 是小智在板端的「编排脚本」，由 `rcS` 在开机时后台执行。典型顺序为：启动 `wifi_manager` 完成 STA 联网 → `amixer` 设置播放/采集音量 → 后台启动 `arecord`、`aplay`、`control_center`、`lvgl_demo`。若脚本中某一行被注释或二进制未编入镜像，表现会是「只有部分功能正常」：例如有 `lvgl_demo` 界面但无 `Opus` 上行，或有联网但无 `TTS` 回放。联调时应 `ps` 确认五个相关进程均存在。

3.6 leds.c 与 GPIO 电平约定

`leds_init` 打开 `/dev/gpio0` (LED1) 与 `/dev/gpio2` (LED2)，通过 `GPIOC_SETPIN` 设为输出。`leds_ctl(0, true)` 对 LED1 写低电平点亮，`leds_ctl(0, false)` 写高电平熄灭，与基础实验 `MyWhackMole` 中 `flash_led1` 一致。进阶实验在 `STT` 分支直接调用该接口，无需再操作寄存器。

3.7 编译打包与 usrdata

实验三应用编入镜像需经过：`m` 交叉编译 NuttX 与用户态程序 → `nuttX` 转 `nuttX.bin` → `pack` 将内核、`rootfs`、`usrdata` 等打包为 `img`。`xiaozhi.sh` 与语音相关资源位于 `UDISK`，`pack` 时打入 `usrdata.fex`，烧录后映射为 `/data/`。若只 `m` 不 `pack`，或 `UDISK` 未更新，板端仍是旧脚本。

四、实验内容与操作步骤

本章按 `示例.txt` 大纲（一）～（六）逐步记录：基础程序检查 → 语音流程 → LED 进阶 → 编译打包 → 实机截图。每一步写清操作顺序、命令、我观察到的现象与排错方法。

4.1 （一）基础程序检查与配置确认

步骤 1：查看资料包 9-5_系统集成。我先打开综合实验资料包 9-5_系统集成，确认包含 `control_center`、`wifi_manager`、`lvgl_demo`、`sound`、`UDISK`、`init.d`、`defconfig` 等，说明课程已给出完整框架（与示例第 26 行一致）。

资料包典型内容：`apps/examples/control_center/`（含 `control_center.c`、`leds.c`）、`wifi_manager`、`lvgl_demo`、`board/common/data/UDISK/xiaozhi.sh`、板级 `defconfig` 与 `rcS`。缺 `xiaozhi.sh` 则只能手工启动各进程，极易漏 `arecord/aplay`。

步骤 2：核对远端 SDK 源码。检查 `~/vela-opensource/apps/examples/` 下 `control_center`、`lvgl_demo`、`wifi_manager`。源码已存在：`control_center.c` 含连接服务器、语音消息、IoT 消息处理；`leds.c` 已封装 LED1/LED2 GPIO（示例第 27 行）。我重点阅读：① 连接后 hello 与 IoT 描述；② Opus 上行循环；③ `process_other_json` 对 `stt/tts/iot` 的分支。原 `iot` 分支已能 `leds_ctl`；题目要求「直接说开灯/关灯」，故在 `stt` 分支加 `strstr` 即可。

步骤 3：发现配置停在实验二。`defconfig` 仍 `CONFIG_EXAMPLES_MYWHACKMOLE=y`，`CONTROL_CENTER/LVGLDEMO` 未启用，`rcS` 中 `sh /data/xiaozhi.sh &` 被注释（示例第 28 行）。不能认为基础任务已配好，须切回实验三配置。

```
grep -E 'MYWHACKMOLE|CONTROL_CENTER|LVGLDEMO' \  
~/vela-opensource/vendor/allwinnertech/boards/r528/r528s3-velaevb1/  
configs/nsh/defconfig
```

步骤 4：切换 `defconfig` 与 `rcS`。将 9-5_系统集成 的 `defconfig` 覆盖到 `vendor/.../nsh/defconfig`，恢复 `rcS` 末尾 `sh /data/xiaozhi.sh &`（示例第 29 行）。切换后 `CONTROL_CENTER`、`LVGLDEMO`、`WIFI_MANAGER`、Wi-Fi、Opus、WebSocket、GPIO、Audio 启用，`MYWHACKMOLE` 关闭。

配置项	目标	说明
<code>CONFIG_EXAMPLES_MYWHACKMOLE</code>		关闭打地鼠
<code>CONFIG_EXAMPLES_CONTROL_CENTER</code>		小智协议枢纽
<code>CONFIG_EXAMPLES_LVGLDEMO</code>	y	激活码与对话 UI
<code>rcS</code> 末尾 <code>xiaozhi.sh</code>	启用	开机自启动

注意

综合实验最容易踩坑的是以为源码存在就等于功能可用。务必用 `menuconfig` 或对比 `defconfig` 确认符号，并在 `pack` 日志中检查 `usrdata` 是否包含 `xiaozhi.sh`。

配置切换操作记录：我将 9-5_系统集成 中的 `defconfig` 覆盖到 `vendor/allwinnertech/boards/r528/r528s3-velaevb1/configs/nsh/defconfig`，并在板级 `init.d/rcS` 中取消注释 `sh /data/xiaozhi.sh &`。覆盖前用 `grep MYWHACKMOLE` 确认实验二符号存在；覆盖后再次 `grep`，确认 `CONFIG_EXAMPLES_CONTROL_CENTER=y`、`CONFIG_EXAMPLES_LVGLDEMO=y`，且 `MYWHACKMOLE` 已关闭。若只改源码不覆盖 `defconfig`，烧录后仍是打地鼠镜像，屏幕不会出现小智激活码。

切换完成后，我用 menuconfig 抽查几项关键符号，与资料包 defconfig 保持一致，避免手工改漏：

```
./build.sh vendor/allwinnertech/boards/r528/r528s3-velaevb1/configs/nsh/ menuconfig
```

在 Application Configuration → Examples 中确认 CONTROL_CENTER、LVGLDEMO、WIFI_MANAGER 均为 [*]；在 Networking 中确认 Wi-Fi STA 相关选项已打开。保存后 grep CONFIG_EXAMPLES 应不再出现 MYWHACKMOLE=y。rcS 修改位置通常在 nuttx/boards/.../init.d/rcS 或板级 overlay 目录，以 SDK 实际路径为准；关键是末尾存在未注释的 sh /data/xiaozhi.sh &。

4.2（二）小智语音助手基础流程实现

步骤 1：理解 xiaozhi.sh 启动链。系统启动后 rcS 执行 /data/xiaozhi.sh，依次启动 wifi_manager、设置音量、arecord、aplay、control_center、lvgl_demo（示例第 31 行）。网络、音频、通信、界面同时工作，无需手工逐个运行。

步骤 2：WebSocket 与 IoT 描述。control_center 连接小智服务器后发 hello，上报 LED1/LED2 的 SetStatus 能力（示例第 32 行）。服务器据此知道板端可控设备。

步骤 3：语音问答数据流。用户讲话 → arecord 采集 Opus → control_center 上传 → 服务器 ASR/LLM/TTS → 下行音频与 JSON → aplay 播放、lvgl_demo 刷新（示例第 33 行）。

程序	职责与联调关注点
wifi_manager	读 wifi.cfg，拉起 wlan0；无 IP 则 WebSocket 失败
arecord	PCM → Opus；串口应有 opus_encode 日志
aplay	TTS 播放；无声先查 amixer
control_center	WebSocket 枢纽；hello、上行、解析 stt/tts/iot
lvgl_demo	激活码与对话 UI

control_center 启动后会通过 WebSocket 连接小智服务器，连接成功后发送 hello 消息，并上报 LED1、LED2 等 IoT 设备描述信息。服务器收到后，就知道开发板端有哪些可控制设备以及它们支持的 SetStatus 方法。

用户讲话时，arecord 负责采集并编码音频，control_center 负责上传音频数据；服务器完成 ASR、LLM、TTS 处理后，再把回复音频和文字状态返回给开发板。aplay 播放音频，lvgl_demo 显示对话状态，形成完整的语音助手交互流程。

联调前自检：上电后执行 `ps`，应能看到 `wifi_manager`、`arecord`、`aplay`、`control_center`、`lvgl_demo` 相关进程（具体名称以 SDK 为准）。若只有 `lvgl_demo` 而无 `arecord`，串口不会有 `Opus` 计数；若只有 `arecord` 而无 `control_center`，音频无法上传。也可在 NSH 中手工执行 `sh /data/xiaozhi.sh &` 复现启动链，逐项确认。

Wi-Fi 配置（若未预置）：可在串口写入热点信息并拉起 `wlan0`：

```
nsh> echo 'ssid=你的热点名' > /data/wifi.cfg
nsh> echo 'psk=你的密码' >> /data/wifi.cfg
nsh> ifup wlan0
nsh> ifconfig wlan0
```

`ifconfig wlan0` 显示非 `0.0.0.0` 的 IP 后，`control_center` 才能向公网发起 WebSocket。本实验连接手机热点，与图 4-1 一致。

`xiaozhi.sh` 启动顺序（板端 `/data/xiaozhi.sh`）：脚本由 `pack` 从 UDISK 打入 `usrdata`，开机由 `rcS` 后台执行。典型内容为：先启动 `wifi_manager` 完成 STA 关联；`amixer` 设置播放/采集音量，避免 TTS 过小或麦克风增益不足；再依次后台拉起 `arecord`（采集 + `Opus`）、`aplay`（播放）、`control_center`（WebSocket 枢纽）、`lvgl_demo`（界面）。若脚本中某一 `&` 行被注释，对应功能会「半残」——例如有界面无上行、有上行无声音。联调时可在 NSH 手工 `cat /data/xiaozhi.sh` 核对内容是否与资料包一致。

WebSocket 握手简述：`control_center` 连上服务器后发送 `type: hello` 的 JSON，携带设备能力与协议版本；服务器应答后，板端上报 IoT 设备描述（LED1/LED2 的 `SetStatus` 能力）。此后二进制 `Opus` 帧与文本 JSON 在同一连接上复用。串口若出现连接失败，先查 IP 与 DNS，再查是否已在 `xiaozhi.me` 完成激活。

4.3（三）进阶功能：语音控制 LED

步骤 1：LED 底层。`leds_init` 打开 `/dev/gpio0`、`/dev/gpio2` 设为输出；`leds_ctl` 控亮灭。LED1 写 0 点亮、写 1 熄灭（示例第 35 行）。

步骤 2：IoT 路径。服务器返回 `type: iot` 且 `commands` 含 LED1 `SetStatus` 时，`control_center` 解析后 `leds_ctl`（示例第 36 行）。

步骤 3：增加 STT 本地判断。在 `control_center.c` 增加 `process_voice_led_command`，对 STT 文本匹配开灯/关灯关键词（示例第 37~52 行）：

```
static void process_voice_led_command(const char *text)
{
    if (text == NULL) {
        return;
    }
}
```

```

    if (strstr(text, "开灯") || strstr(text, "打开灯") ||
        strstr(text, "打开LED") || strstr(text, "打开LED1")) {
        printf("voice command: turn on LED1\n");
        leds_ctl(0, true);
    } else if (strstr(text, "关灯") || strstr(text, "关闭灯") ||
        strstr(text, "关闭LED") || strstr(text, "关闭LED1")) {
        printf("voice command: turn off LED1\n");
        leds_ctl(0, false);
    }
}

```

步骤 4: 在 stt 分支挂接。 process_other_json 的 stt 分支里，原逻辑仅 send_stt(text->valuestring) 送界面。我在其后调用 process_voice_led_command(text->valuestring)，识别文本一到板端即可控灯，无需唤醒词，不影响 TTS（示例第 53 行）。

关键词列表：开灯类——「开灯」「打开灯」「打开 LED」「打开 LED1」；关灯类——「关灯」「关闭灯」「关闭 LED」「关闭 LED1」。命中后串口应打印 voice command: turn on LED1 或 turn off LED1，与板载 LED1 亮灭一致（动态过程见视频附件）。

与示例大纲对应：示例要求当 type 为 stt 时，在 send_stt(text->valuestring) 之后调用 process_voice_led_command(text->valuestring)，识别文本一到板端即可控灯，不需要额外唤醒词，也不影响原本的 TTS 对话流程。联调时若对话正常但灯不亮，应在串口搜索 voice command；若无该日志，说明 STT 分支未挂接或关键词未命中（ASR 可能识别为同音字，可临时 printf 打印原始 text 排查）。

4.4（四）编译、打包和结果确认

步骤 1: 重新编译。在 ~/vela-opensource/vendor/allwinnertech/lichee/ 执行 m。日志中应见 control_center、lvgl demo、wifi_manager 注册进系统，说明实验三应用已进入镜像（示例第 55 行）。

步骤 2: 打包。nuttx 转 nuttx.bin，复制到 board 配置参与 pack；usrdata.fex 应含 xiaozhi.sh，生成 rtos_nuttx_r528s3-velaevb1_uart0_256Mnand.img（示例第 56 行，约 39 MB）。

步骤 3: ap.fex 提示。打包末尾可能出现 ap.fex 不存在，但若已有 Dragon SUCCESS、pack finish 且 out 目录有约 39 MB 的 img，则本次镜像有效（示例第 57 行）。PhoenixSuit 烧录后应自启 xiaozhi.sh，而非 mywhackmole。

```
cd ~/vela-opensource/vendor/allwinnertech/lichee/
source vela_env.sh
source envsetup.sh
lunch_nutttx
m
pack
```

1. 打包流程：编译完成后，将 nuttx 转换为 nuttx.bin，并复制到 board 配置目录参与 pack，生成最终镜像。pack 过程中可以看到 usrdata.fex 中已经包含 xiaozhi.sh 和相关资源文件，最后生成 rtos_nutttx_r528s3-velaevb1_uart0_256Mnand.img 镜像。与示例大纲一致，pack 大致经历：① m 生成 nuttx 可执行文件；② 工具链将 nuttx 转为 nuttx.bin；③ 复制到 board/r528s3/velaevb1_nand/configs/nsh.fex 参与打包（路径以 SDK 为准）；④ pack 合并内核、rootfs、usrdata 等为单一 img。pack 日志中若出现 usrdata.fex 且列出 xiaozhi.sh，说明 /data 分区内容正确。最终产物位于 out/r528s3/velaevb1_nand/，文件名 rtos_nutttx_r528s3-velaevb1_uart0_256Mnand.img，本实验约 39 MB。
1. 打包异常说明：打包脚本末尾出现 ap.fex 不存在的提示，但前面已经显示 Dragon execute image.cfg SUCCESS 和 pack finish，并且 out 目录下生成了约 39 MB 的 img 文件，所以本次镜像生成有效。PhoenixSuit 烧录后应自动执行 xiaozhi.sh，而非实验二的 mywhackmole。

现象	可能原因与处理
无自启动	rcS 或 usrdata 未含 xiaozhi.sh
有 UI 无 Opus	arecord 未启动；ps 查进程
WebSocket 失败	Wi-Fi 或激活未完成
能对话灯不亮	STT 分支未调用 process_voice_led_command
TTS 无声	aplay 或 amixer 音量

4.5（六）相关实验截图与实机联调

烧录成功后，我按下列顺序实机操作（与示例第六、七节一致）：

- ① 配置 Wi-Fi。在串口写入热点信息并 ifup wlan0（连的是自己的热点）。串口出现 Opus 编码与 Send opus data to server，说明上行正常（图 4-1）。
- ② 屏幕激活码。连上热点后 lvgl demo 显示激活码（图 4-2）。
- ③ 手机激活。浏览器打开 <https://xiaozhi.me>，输入板载激活码（图 4-3）。
- ④ 激活成功。手机提示绑定成功（图 4-4）。

- ⑤ 语音对话。 对着麦克风提问，板端 TTS 播放、界面刷新（图 4-5）。
 - ⑥ 流程图。 整理端到端流程图，与 § 五 一致（图 4-6）。
- 注意： 开灯、关灯动态演示见视频附件（示例第 78 行）。

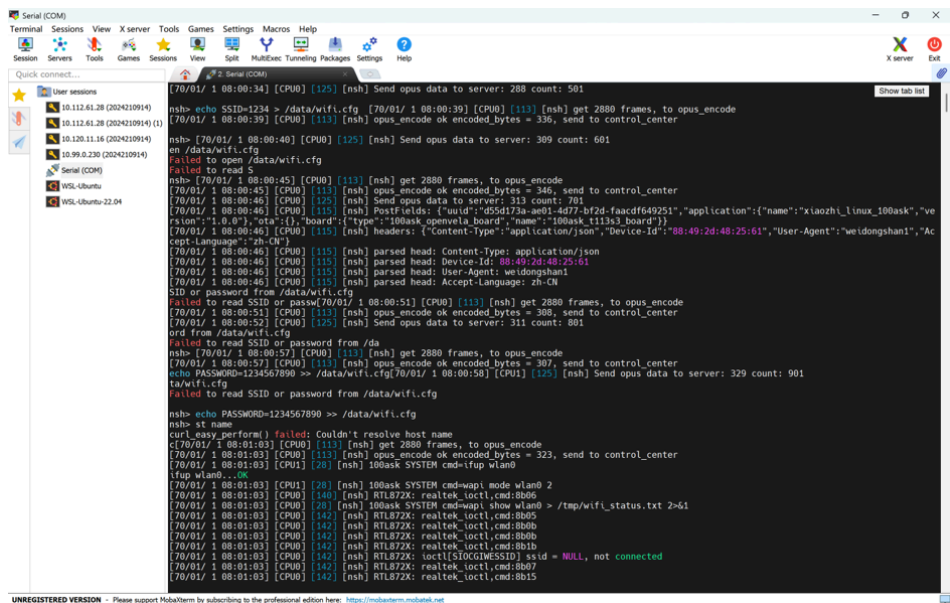


Figure 1: 图 4-1 串口 Wi-Fi 配置与 Opus 音频上行日志

图 4-1: 串口可见 Wi-Fi 关联与 get ... frames, to opus_encode、Send opus data to server 等日志，说明麦克风采集 → Opus → 上行链路正常；若无此类输出，优先检查 arecord 与 xiaozhi.sh 是否执行。板子成功连接上热点后，屏幕显示了激活码。我使用手机登录网址 <https://xiaozhi.me>，输入激活码准备激活小智。

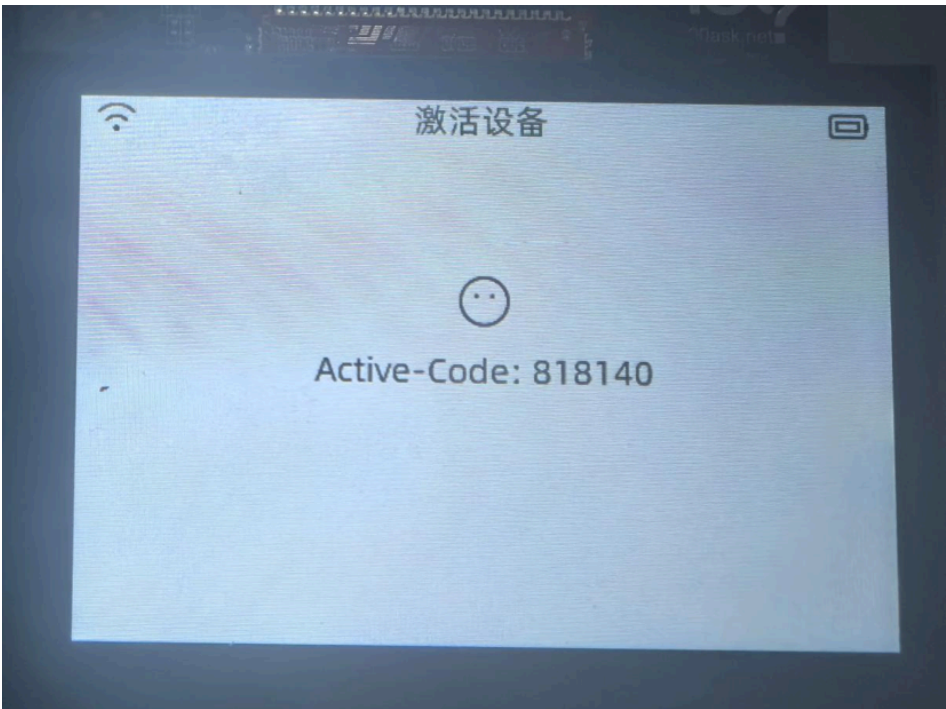


Figure 2: 图 4-2 开发板屏幕显示激活码

图 4-2: lvgl_demo 界面显示设备激活码，需在手机端 xiaozi.me 输入以完成绑定。



Figure 3: 图 4-3 手机端 xiaozihi.me 激活页面

图 4-3： 手机浏览器打开 xiaozhi.me 激活页，输入板载屏幕上的激活码。
如下图所示，小智激活成功。



Figure 4: 图 4-4 小智设备激活成功

图 4-4：手机端提示激活成功，此后 WebSocket 鉴权可通过，control_center 可正常连服务器。
然后就可以用板子和小智对话。对着麦克风提问后，板端播放 TTS 回复，界面同步显示识别与回答内容。

```
Device has been Activated
[1970/01/01 08:06:00:9268] N: lws_create_context: LWS: 4.3.1-unknown, NET CLI H1 H2 WS MbedTLS ConMon IPv6-absent
[1970/01/01 08:06:00:9290] N: lws_lc_tag: ++ [vsi0|pipe] (1)
[1970/01/01 08:06:00:9290] N: lws_lc_tag: ++ [vsi0|default] (1)
[1970/01/01 08:06:00:9290] N: lws_lc_tag: ++ [vsi0|WS/h1/default/api.tenclass.net] (1)
[70/01/ 1 08:06:00] [CPU1] [1517] [nsh] ERROR: Failed to open the hosts file /tmp/hosts: -2
[70/01/ 1 08:06:00] [CPU0] [1517] [nsh] Connecting to wss://api.tenclass.net:443/xiaozi/v1/
[70/01/ 1 08:06:00] [CPU0] [1517] [nsh] begin lws_service
[1970/01/01 08:06:00:9880] N: lws_ssl_client_bio_crea[70/01/ 1 08:06:01] [CPU0] [1517] [nsh] Add header: Authorization: Bearer test-token
[70/01/ 1 08:06:01] [CPU0] [1517] [nsh] Add header: Protocol-Version: 1
[70/01/ 1 08:06:01] [CPU0] [1517] [nsh] Add header: Device-Id: 88:49:2d:48:25:61
[70/01/ 1 08:06:01] [CPU0] [1517] [nsh] Add header: Client-Id: d55d173a-ae01-4d77-bf2d-faacdf649251
[70/01/ 1 08:06:01] [CPU1] [1517] [nsh] Connected
[70/01/ 1 08:06:01] [CPU1] [1517] [nsh] Send hello message: {"type":"hello","version":1,"transport":"websocket","audio_params":{"format":"opus","sample_rate":16000,"channels":1,"frame_duration":60}}
[70/01/ 1 08:06:01] [CPU1] [28] [nsh] 100ask SYSTEM cmd=ifup wlan0
ifup wlan0...OK
[70/01/ 1 08:06:01] [CPU1] [28] [nsh] 100ask SYSTEM cmd=wapi mode wlan0 2
[70/01/ 1 08:06:01] [CPU1] [1524] [nsh] RTL872X: realtek_ioctl,cmd:8b06
[70/01/ 1 08:06:01] [CPU1] [28] [nsh] 100ask SYSTEM cmd=wapi show wlan0 > /tmp/wifi_status.txt 2>61
[70/01/ 1 08:06:01] [CPU1] [1540] [nsh] RTL872X: realtek_ioctl,cmd:8b05
[70/01/ 1 08:06:01] [CPU1] [1540] [nsh] RTL872X: realtek_ioctl,cmd:8b0b
[70/01/ 1 08:06:01] [CPU1] [1540] [nsh] RTL872X: realtek_ioctl,cmd:8b0b
[70/01/ 1 08:06:01] [CPU1] [1540] [nsh] RTL872X: realtek_ioctl,cmd:8b1b
[70/01/ 1 08:06:01] [CPU1] [1540] [nsh] RTL872X: realtek_ioctl,cmd:8b07
```

Figure 5: 图 4-5 板端与小智语音对话运行界面

图 4-5：对话进行中，屏幕显示识别/回复状态，扬声器可听到 TTS；说「开灯」「关灯」时串口应出现 voice command 日志。
本次实验整理的「小智语音通信端到端执行流程图」如下，与 § 五 文字描述一致。

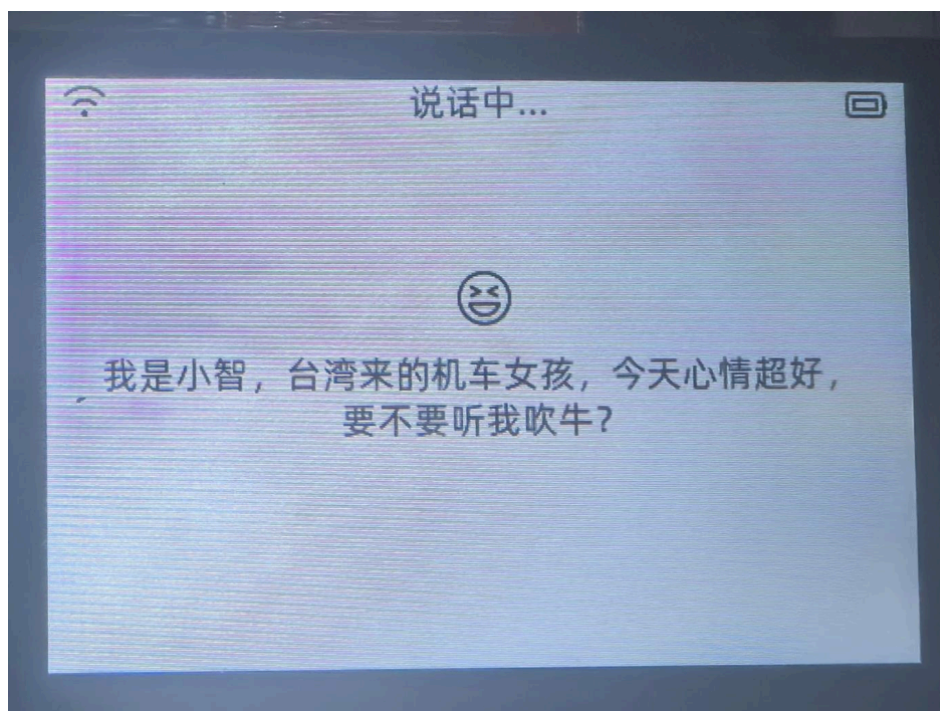


Figure 6: 图 4-6 小智语音通信端到端执行流程图

图 4-6：手绘/设计稿流程图：用户语音 → 板端 Opus 上行 → 服务器 ASR/LLM/TTS → 板端播放；支路为 STT 文本触发 LED1 GPIO。

4.6 联调分段检查清单

综合实验建议按下列顺序分段确认，避免「一上来就查 LED 代码」而忽略配置与启动链：

阶段	检查命令/现象	通过标准
① 镜像	grep defconfig; pack 日志	CONTROL_CENTER=y; usrdata 含 xiaozhi.sh
② 自启动	复位后 ps	xiaozhi.sh 拉起多进程
③ 网络	ifconfig wlan0	已获取 IP
④ 上行	串口 Opus 日志	Send opus data 计数递增
⑤ 激活	屏幕激活码; xiaozhi.me	绑定成功
⑥ 对话	麦克风提问	TTS 可听; UI 刷新
⑦ LED	说开灯/关灯	串口 voice command; LED1 亮灭

任一阶段失败，应先解决该阶段再进入下一阶段；这与基础实验「先 fb 再改驱动」、初探实验「先 hello 再 pthread」的分段思路一致。

注意：开灯、关灯的动态演示见视频附件。联调时若对话正常但灯不亮，应在串口搜索 voice command 关键字，确认 STT 分支已挂接 process_voice_led_command；若仅有 IoT 路径日志而无 voice command，说明用户话术未命中本地关键词，可对照 ASR 返回的原始文本调整 strstr 列表。

五、端到端流程图

下图概括用户一次完整语音交互的数据路径。上行以 Opus 帧经 WebSocket 发送；下行除 TTS 音频外，STT 文本在板端可触发 LED 支路，与云端 IoT 命令并行存在。

开发板 ←WebSocket→ 小智服务器 ←API→ ASR / LLM / TTS

用户 TTS	开发板(T113)	小智服务器	ASR	LLM
	-- 说话 ----->			
		-- Opus WS ----->		
		-- 音频 ----->		
		<-- 文本 -----		
		-- 用户文本 ----->		
		<-- 回复文本 -----		
		-- 回复文本		
	----->			
		<-- 音频流		
	<-- 音频 WS -----			
	<-- 扬声器播放 -			
	[STT 含开灯/关灯]			
	-- GPIO LED1 ---			

Listing 1: 图 5-1 小智语音通信端到端流程（含 LED 控制支路）

图中上行以 Opus 帧为单位，下行除 TTS 音频外还包含 stt/tts 状态 JSON；进阶支路在 STT 文本到达板端时本地分叉，不必等待 LLM 完整回复即可控灯。

流程图文字说明：从用户讲话开始，开发板 arecord 采集并经 Opus 编码，control_center 经 WebSocket 上传至小智服务器；服务器 ASR 得文本 → LLM 生成回复 → TTS 合成语音，再经同一连接回传；aplay 播放，lvgl demo 刷新界面。进阶支路：服务器下发 type: stt 的 JSON 时，板端除显示文本外，本地 process_voice_led_command 解析「开灯/关灯」并驱动 GPIO；云端 type: iot 的 SetStatus 路径仍保留，两条路径可并存。

五、端到端流程图说明（对应示例大纲）：本次实验另生成「小智语音通信端到端执行流程图」（§ 4.5 图 4-6），与上图 ASCII 版一致，便于答辩展示完整链路与 LED 支路。

六、关键源码、烧录流程与排错记录

6.1 xiaozhi.sh 自启动脚本（板端 /data/xiaozhi.sh）

rcS 末尾 `sh /data/xiaozhi.sh &` 拉起下列进程；缺任一行会出现「能亮屏不能对话」等半残状态：

```
#!/bin/sh
# 板端 /data/xiaozhi.sh (由 pack 打入 usrdata)
wifi_manager &
amixer set 6 180
amixer set 7 180
amixer set 15 7
arecord &
aplay &
control_center &
lvgl_demo &
```

6.2 STT 分支挂接与语音控灯函数

`process_voice_led_command`（新增）：

```
static void process_voice_led_command(const char *text)
{
    if (text == NULL) {
        return;
    }
    if (strstr(text, "开灯") || strstr(text, "打开灯") ||
        strstr(text, "打开LED") || strstr(text, "打开LED1")) {
        printf("voice command: turn on LED1\n");
        leds_ctl(0, true);
    } else if (strstr(text, "关灯") || strstr(text, "关闭灯") ||
        strstr(text, "关闭LED") || strstr(text, "关闭LED1")) {
        printf("voice command: turn off LED1\n");
        leds_ctl(0, false);
    }
}
```

在 `process_other_json` 的 `stt` 分支挂接（`send_stt` 之后）：

```

/* 实验三: process_other_json 中 stt 分支挂接 LED 控制 */
} else if (strcmp(type->valuestring, "stt") == 0) {
    cJSON *text = cJSON_GetObjectItem(root, "text");
    if (text != NULL && cJSON_IsString(text)) {
        send_stt(text->valuestring);          /* 原逻辑: 送 lvgl demo 显示 */
        process_voice_led_command(text->valuestring); /* 进阶新增: 本地关键词控灯 */
    }
}
}

```

IoT 路径(原有): 服务器下发 type:iot 且 commands 含 LED1 SetStatus 时, leds_ctl 同样可亮灯; STT 本地路径响应更快, 两条并存。

6.3 LED 底层 leds.c

```

/* leds.c: LED1 低电平点亮 (与 MyWhackMole flash_led1 一致) */
int leds_ctl(int led, bool on)
{
    if (led == 0) {
        /* LED1 → /dev/gpio0, 写 0 亮, 写 1 灭 */
        return gpio_write(led1_fd, on ? 0 : 1);
    }
    return -1;
}

```

6.4 defconfig 与 rcS 切换命令

```

grep -E 'MYWHACKMOLE|CONTROL_CENTER|LVGLDEMO' \
    vendor/allwinnertech/boards/r528/r528s3-velaevb1/configs/nsh/
defconfig

```

```

# 切换实验三: 用 9-5_系统集成 的 defconfig 覆盖
# rcS 取消注释:
# sh /data/xiaozhi.sh &

```

```

# 板级 init.d/rcS 与本实验相关的两行 (路径以 SDK 为准)

```

```
# 实验一：若串口被综合实验 xiaozhi 刷屏，注释掉：
/* sh /data/xiaozhi.sh & */

# 实验一可选：开机自启动 hello (须重新 m && pack):
hello &

# 实验三：恢复小智自启动 (切换 defconfig 后取消注释):
sh /data/xiaozhi.sh &
```

6.5 编译、打包、烧录

```
#!/bin/bash
# openvela 标准编译、打包、定位镜像 (三项实验通用)
cd ~/vela-opensource/vendor/allwinnertech/lichee/
source vela_env.sh
source envsetup.sh
lunch_nuttx      # 选 2: r528s3-velaevb1
m
pack
find ~/vela-opensource/vendor/allwinnertech/lichee/out \
    -name "rtos_nuttx_r528s3-velaevb1_uart0_256Mnand.img"

# Windows: 将 img 拷到本机, PhoenixSuit → 一键刷机 → 全盘擦除升级
# 按住 FEL, 短按 RST, 等待烧录完成
# MobaXterm: Serial, 1500000, Flow Control = none
```

pack 日志应含 usrdata.fex 与 xiaozhi.sh。末尾 ap.fex 不存在但 pack finish 且 img 约 39 MB 可视为成功。

6.6 问题现象、原因与修改

现象	原因	修改
仍是打地鼠	defconfig 未切	覆盖 9-5 defconfig; 关 MYWHACKMOLE
无 xiaozhi 自启	rcS 注释	恢复 sh /data/xiaozhi.sh &
无 0pus 日志	arecord 未起	ps 查进程; 检查 xiaozhi.sh 是否进 usrdata

现象	原因	修改
WebSocket 失败	未联网/未激活	wifi.cfg + ifup; xiaozhi.me 绑激活码
能对话灯不亮	stt 未挂接	send_stt 后加 process_voice_led_command
无 voice command	关键词未命中	printf 原始 STT 文本; 扩展 strstr 列表
TTS 无声	aplay/ amixer	xiaozhi.sh 内 amixer; 单独 aplay 测试
ap.fex 报错	打包脚本提示	以 pack finish 与 img 大小为准, 可忽略

七、实验结果汇总

任务	完成	说明
资料包与源码核对	✓	9-5_系统集成 框架完整; SDK examples 源码齐全
defconfig / rcS 切换	✓	关闭 WhackMole; 启用语音套件与 xiaozhi.sh 自启动
编译注册	✓	m 日志可见 control_center、lvgl_demo、wifi_manager
pack / usrdata	✓	usrdata 含 xiaozhi.sh; img 约 39 MB
Wi-Fi 联网	✓	STA 获取 IP; Opus 上行计数正常 (图 4-1)
云端激活	✓	xiaozhi.me 绑定成功 (图 4-2~4-4)
语音问答	✓	ASR→LLM→TTS 全链路可听可见 (图 4-5)
端到端流程图	✓	§ 五 与图 4-6
语音开灯	✓	STT 关键词 → leds_ctl; 串口 voice command
语音关灯	✓	同上

常见故障与排查:

现象	可能原因与处理
烧录后仍像实验二	defconfig 未切换; grep MYWHACKMOLE
无 xiaozhi 自启动	rcS 注释或 usrdata 无脚本; 查 pack 日志
有 UI 无 Opus 日志	arecord 未起; ps 与 xiaozhi.sh
WebSocket 连不上	Wi-Fi 未通或未激活; 查 IP 与激活码
能对话灯不亮	stt 分支未挂 process_voice_led_command
TTS 无声	aplay/amixer; 单独 aplay 测试
ap.fex 报错但 img 正常	可忽略; 以 pack finish 与 img 大小为准

八、实验总结

通过本次综合实验，我对小智 AI 语音助手在开发板上的整体运行方式有了更清楚的理解。它不是单独一个程序完成所有事情，而是由 Wi-Fi 管理、录音、播放、服务器通信和 LVGL 界面多个部分配合完成；任何一环未启动，表现都会是「能亮屏但不能对话」或「能联网但没声音」。

基础任务回顾：课程资料包 9-5_系统集成 已给出完整框架，但远端工程曾停留在实验二配置——CONFIG_EXAMPLES_MYWHACKMOLE 仍开启，control_center 与 lvgl_demo 未编入镜像，rcS 自启动被注释。我通过替换 defconfig、恢复 `sh /data/xiaozhi.sh &` 并完成 `m && pack`，使语音助手真正进入系统。这一过程说明：综合项目里「源码在仓库里」不等于「功能在板子上」，配置与启动脚本与源码同等重要。

进阶任务回顾：在原有 IoT_SetStatus 路径之外，我在 STT 分支增加 process_voice_led_command，对用户说出的「开灯」「关灯」等关键词做本地 strstr 匹配，直接调用 leds_ctl(0, true/false)。功能代码量不大，但把语音识别结果、WebSocket 消息解析与 GPIO 控制串成闭环，是可演示的语音控外设样例。串口日志 voice command: turn on LED1 是联调成功的直接证据。

个人收获：联调时我优先看三类信号——Opus 帧计数（上行）、WebSocket 连接/激活（云端）、TTS 播放（下行）；LED 问题则看 STT 文本是否命中与 leds_ctl 是否执行。相比在 control_center 里盲目加 printf，按数据流分段排查效率更高。这与初探实验

「先判断是调度饿死还是硬件故障」、基础实验「先区分显示驱动还是触摸映射」的思路一致。

后续可扩展方向：在 IoT 描述中注册更多传感器；用同义词表支持「把灯打开」「灯关一下」等更自然的说法；将 LED 控制从纯 STT 关键词改为 Intent 模块输出，降低误触发；或将 MQTT+UDP 链路替换 WebSocket 以适配不同部署。

这次实验让我感觉综合项目最容易出问题的地方不一定是某一行代码，而是配置、启动脚本、资源路径和多个任务之间有没有真正接起来。后面如果继续完善，可以把更多设备加入 IoT 描述，也可以把语音指令写得更灵活。综合实验让我体会到，嵌入式 AI 应用的难点往往在集成与配置，业务代码往往是最后一块拼图。